IDAT7215
Computer Programming for Product Development and Applications

Lecture 2-1: Python Libraires: NumPy

Dr. Zulfiqar Ali

Recap

- **Introduction to Programming**
- **Python Installation**
- **Python Fundamentals**
  - Variables and Types in Python
  - Data Structures in Python
  - Python Functions
  - Control Flows

Outline

- NumPy Introduction
- Creation of Arrays
- Data Retrieve and Restructure
- Fast Computation with NumPy

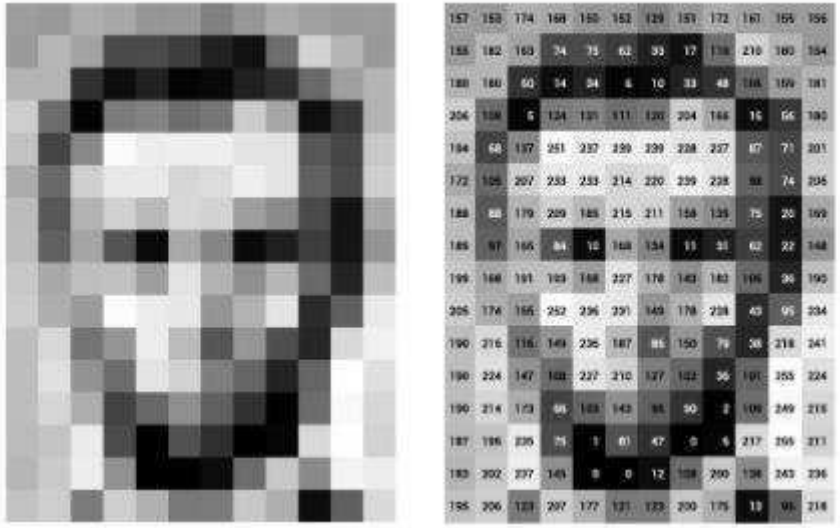
NumPy

- NumPy stands for Numerical Python and it is the fundamental package for scientific computing with Python.
- NumPy is a Python library for handling multi-dimensional arrays.
- It contains both the **data structures** needed for the storing and accessing arrays, and **operations and functions** for computation using these arrays.
- Unlike lists, the arrays must have the same data types for all its elements.
- The **homogeneity** of arrays allows highly optimized functions that use arrays as their inputs and outputs.

Usages of high-dimensional arrays in data analysis

- Store matrices, solve systems of linear equations, compute eigenvalues/eigenvectors, matrix decompositions, …

- Images and videos can be represented as NumPy arrays



Usages of high-dimensional arrays in data analysis

- Store matrices, solve systems of linear equations, compute eigenvalues/eigenvectors, matrix decompositions, …
- Images and videos can be represented as NumPy arrays
- A 2-dimensional table might store an input data matrix in data analysis, where row represents a sample, column represents a feature (Commonly used in scikit-learn).

Creation of arrays

- Import the NumPy library
  - Suggested to use the standard abbreviation `np`

```
import numpy as np
# creation of arrays
a =  np.array([1,2,3]) # one dimensional array
a
```

```
array([1, 2, 3])
```

```
np.array([[1,2,3], [4,5,6]])  # two dimensional array
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
np.array([[[1,2], [3,4]], [[5,6], [7,8]]]) # three dimensional array
```

```
array([[[1, 2],
        [3, 4]],

       [[5, 6],
        [7, 8]]])
```

Creation of arrays

- Import the NumPy library
  - Suggested to use the standard abbreviation `np`
- Give a (nested) list as a parameter to the `array` constructor
  - One dimensional array: list
  - Two dimensional array: list of lists
  - Three dimensional array: list of lists of list

```
import numpy as np
# creation of arrays
a =  np.array([1,2,3]) # one dimensional array
a
```

```
array([1, 2, 3])
```

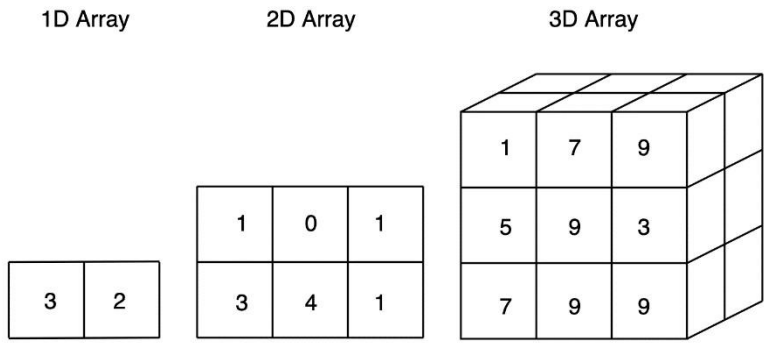
```
np.array([[1,2,3], [4,5,6]])  # two dimensional array
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

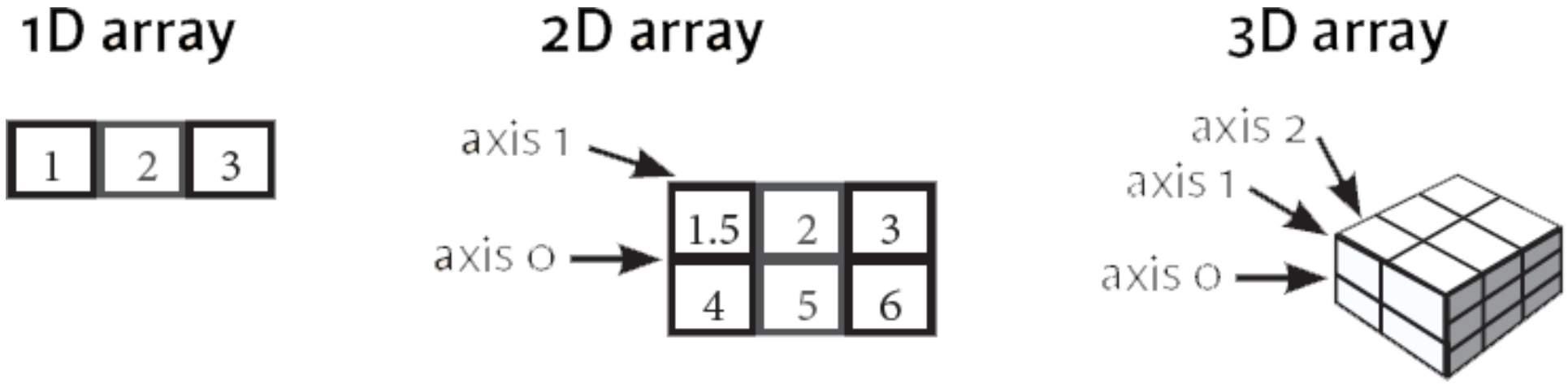
```
np.array([[[1,2], [3,4]], [[5,6], [7,8]]]) # three dimensional array
```

```
array([[[1, 2],
        [3, 4]],

       [[5, 6],
        [7, 8]]])
```



One dimensional array, Two dimensional array, Three dimensional array



- In a two-dimensional array, you have rows and columns. The rows are indicated as “axis 0” while the columns are the “axis 1”.
- The number of the axis goes up accordingly with the number of the dimensions.

Creation of arrays

- Useful function to create common types of arrays
  - `np.zeros()`: all elements are 0s
  - `np.ones()`: all elements are 1s
  - `np.full()`: all elements to a specific value
  - `np.empty()`: all elements are uninitialized
  - `np.eye()`: identity matrix: a matrix with elements on the diagonal are 1s, others are 0s

```
# create common types of arrays:
np.zeros((3,4))
```

```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

```
np.ones((3,3))
```

```
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
```

```
np.full((3,3), fill_value=3.0)
```

```
array([[3., 3., 3.],
       [3., 3., 3.],
       [3., 3., 3.]])
```

```
np.empty((2,2))
```

```
array([[1.19956159e-311, 2.13620807e-306],
       [3.44897822e-307, 6.89794965e-307]])
```

```
np.eye(5)
```

```
array([[1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 1.]])
```

Creation of arrays

```
np.arange(1,10,3)  # the start value is 1 (inclusive), stop value is
```

```
array([1, 4, 7])
```

```
np.linspace(0, np.pi, 5)  # Evenly spaced range with 5 elements
```

```
array([0.        , 0.78539816, 1.57079633, 2.35619449, 3.14159265])
```

- Generate evenly spaced values within a given interval.
- `np.arrange()`: It works like Python built-in `range()` function
- For non-integer ranges it is better to use `np.linspace()`.
- With `np.linspace()` one does not have to compute the length of the step, but instead one specifies the wanted number of elements. By default, the endpoint is included in the result, unlike with arange.

Creation of arrays with random elements

- We may need some random generated data to test our program
- NumPy can easily produce arrays of wanted shape with random numbers.
- `np.random.random()`: uniformly distributed from [0.0, 1.0)
- `np.random.normal()`: normally distributed
- `np.random.randint()`: uniformly distributed integers

Creation of arrays with random elements

```
# Creation of arrays with random elements
np.random.random((3,4)) # Elements are uniformly distributed from [0.0,1.0)
```

```
array([[0.22051398, 0.7076659 , 0.42798583, 0.01112018],
       [0.97417167, 0.26964842, 0.36510724, 0.28772977],
       [0.04420915, 0.51836288, 0.92733042, 0.55925031]])
```

```
np.random.normal(0, 1, (3,4)) # Elements are normally distributed with mean 0 and standard deviation 1
```

```
array([[-1.25509693,  1.25917427,  0.16378993, -0.3458001 ],
       [ 0.52594032,  0.55853451, -0.56651307,  1.20601537],
       [-0.2643899 ,  1.18457211, -0.32818969, -0.06231104]])
```

```
np.random.randint(-2, 10, (3,4))  # Elements are uniformly distributed integers from [-2,10)
```

```
array([[-1,  0,  8, -2],
       [ 3,  5,  3,  9],
       [ 1,  8,  9,  8]])
```

Creation of arrays with random elements

- To debug our code, sometimes it is useful to re-create exactly the same random data in every run of our program.

- We can create random numbers deterministically using `seed`.

```
# create random numbers deterministically using seed.
np.random.seed(0)
print(np.random.randint(0, 100, 10))
```

```
[44 47 64 67 67  9 83 21 36 87]
```

If you run the code multiple times, it will always give the same numbers,

Array types and attributes

- An array has several attributes:
  - `ndim`: the number of dimensions
  - `shape`: size in each dimension
  - `size`: the number of elements
  - `dtype`: the type of element

```
b=np.array([[1,2,3], [4,5,6]])
```

```
print(b.ndim)
print(b.shape)
print(b.size)
print(b.dtype)
```

```
2
(2, 3)
6
int32
```

```
b=np.array([[[1,2,3], [4,5,6]], [[1,2,3], [4,5,6]]])
print(b.ndim)
print(b.shape)
print(b.size)
print(b.dtype)
```

```
3
(2, 2, 3)
12
int32
```

Indexing

- One dimensional array works like the list.
- For multi-dimensional array, the index is a comma separated tuple instead of single integer
- Note that if you give only a single index to a multi-dimensional array, it indexes the first dimension of the array.

```
# Indexing
a=np.array([1,4,2,7,9,5])
print(a[1])


b=np.array([[1,2,3], [4,5,6]])
print(b)
print(b[1,2])    # row index 1, column index 2

# As with lists, modification through indexing is possible
b[0,0] = 10
print(b)

# if give only a single index to a multi-dimensional array
print(b[0])    # First row
print(b[1])    # Second row
```

```
4
[[1 2 3]
 [4 5 6]]
6
[[10  2  3]
 [ 4  5  6]]
[10  2  3]
[4 5 6]
```

Slicing

- Slicing works similarly to lists, but now we can have slices in different dimensions.
- We can even assign to a slice
- Extract rows or columns from an array

```
# Slicing
a=np.array([1,4,2,7,9,5])
print(a[1:3])

b=np.array([[1,2,3], [4,5,6]])
print(b[:,0])
print(b[0,:])
print(b[:,1:])
```

```
[4 2]
[1 4]
[1 2 3]
[[2 3]
 [5 6]]
```

```
# assign to a slice
b[:,1:] = 7
print(b)
```

```
[[1 7 7]
 [4 7 7]]
```

```
print(b[:,0])    # First column
print(b[1,:])    # Second row
```

```
[1 4]
[4 7 7]
```

Reshaping

- When an array is reshaped, its number of elements stays at the same, but they are reinterpreted into a different shape.

- E.g., one dimensional array into two dimension array

```
# reshape
a=np.arange(9)
anew=a.reshape(3,3)
print(a)
print(anew)
```

```
[0 1 2 3 4 5 6 7 8]
[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

Combining Arrays

- Combining several arrays into on bigger array: `concatenate` and `stack`
- `Concatenate:` It takes n-dimensional arrays and return an n-dimensional array.
- `Stack:` it takes n-dimensional arrays and return (n+1)-dimensional array

```
: # Combining Arrays
  # concatenation
  a = np.array([[1,2],[3,4]])
  b = np.array([[5,6],[7,8]])
  print(a)
  print(b)
  c = np.concatenate((a,b))
  print(a.ndim)
  print(c.ndim)
```

```
[[1 2]
 [3 4]]
[[5 6]
 [7 8]]
2
2
```

```
: # stack
  print(np.stack((a,b)))
  print(np.stack((a,b)).ndim)
```

```
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
3
```

Concatenate

- By default, concatenate joins the arrays along axis 0.

- To joint array horizontally, add parameter `axis = 1`

```
# concatenate
a = np.array([[1,2],[3,4]])
b = np.array([[5,6],[7,8]])
print(a)
print(b)
print(np.concatenate((a,b)))
```

```
[[1 2]
 [3 4]]
[[5 6]
 [7 8]]
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

```
print(np.concatenate((a,b), axis = 1))
```

```
[[1 2 5 6]
 [3 4 7 8]]
```

Concatenate different dimensions

- If you want to concatenate arrays with different dimensions, you must first reshape the arrays to have the same number of dimensions.
  - E.g, add a new row (column) to a 2d array

```
a = np.array([[1,2],[3,4]])
b = np.array([5,6])
print(np.concatenate((a,b)))
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-66-01163ce3ad68> in <module>
      1 a = np.array([[1,2],[3,4]])
      2 b = np.array([5,6])
----> 3 print(np.concatenate((a,b)))

ValueError: all the input arrays must have same number of dimensions
```

```
a = np.array([[1,2],[3,4]])
b = np.array([5,6])
print(np.concatenate((a,b.reshape(1,2))))
```

```
[[1 2]
 [3 4]
 [5 6]]
```

```
print(np.concatenate((a,b.reshape(2,1)), axis = 1))
```

```
[[1 2 5]
 [3 4 6]]
```

Stack

- Use `stack` to create higher dimensional arrays from lower dimensional arrays:

```
b = np.array([4,5,6])
print(np.stack((b,b)))
```

```
[[4 5 6]
 [4 5 6]]
```

Split

- `split` is the inverse operation of `concatenate`.

- The input argument to `split` can be the number of equal parts the arrays is divided into.

```
# Split
d=np.arange(12).reshape(6,2)
print("d:")
print(d)
d1,d2 = np.split(d, 2)
print("d1:")
print(d1)
print("d2:")
print(d2)
```

```
d:
[[ 0  1]
 [ 2  3]
 [ 4  5]
 [ 6  7]
 [ 8  9]
 [10 11]]
d1:
[[0 1]
 [2 3]
 [4 5]]
d2:
[[ 6  7]
 [ 8  9]
 [10 11]]
```

Split

- The input argument to `split` can also be indices that specified explicitly the break points.
- The entries indicate where along axis (default axis = 0) the array is split.
- E.g., `np.split(d, (2, 3, 5))` split array `d` into
  - `d[:2]`
  - `d[2:3]`
  - `d[3:5]`
  - `d[5:]`

```
d=np.arange(12).reshape(6,2)
print("d:")
print(d)
sub_d = np.split(d, (2,3,5))
for d_i in sub_d:
    print(d_i)
```

```
d:
[[ 0  1]
 [ 2  3]
 [ 4  5]
 [ 6  7]
 [ 8  9]
 [10 11]]
[[0 1]
 [2 3]]
[[4 5]]
[[6 7]
 [8 9]]
[[10 11]]
```

Less Memory in NumPy

- Space occupied by NumPy is less compare to list.

```
import numpy as np
import time
import sys
S =range(1000)
print(sys.getsizeof(5)*len(S))

D = np.arange(1000)
print(D.size*D.itemsize)
```

```
28000
4000
```

Fast computation on arrays

- In addition to providing a way to store and access multi-dimension arrays, NumPy also provides several routines to perform computations on them.
- One of the reasons for the popularity of NumPy is that these computations can be very efficient, much more efficient than what Python can normally do.
- The biggest bottle-necks in efficiency are the loops, which can be iterated millions, billions, or even more times.
- What slows down loops in Python is the fact that Python is dynamically typed language: at each expression Python has to find out the types of the arguments of the operations.

Fast computation examples

- Let consider multiply 2 to a collection of numbers.
- At each iteration of the loop, Python has find out the type of the variable x, which can in this example be an int, a float or a string, and depending on this type call a different function to perform the “multiplication” by two.
- What makes NumPy efficient, is the requirement that each element in an array must be of the same type. This homogeneity of arrays makes it possible to create *vectorized* operation, which don’t operate on single elements, but on arrays (or subarrays).

```
# fast computation
def double_in_list(L):
    L2=[]
    for x in L:
        L2.append(x*2)
    return L2

l = range(1000)
#print(double_in_list(l))
%timeit double_in_list(l)

a=np.arange(1000)
#print(a*2)
%timeit a*2
```

```
239 µs ± 32.5 µs per loop (mean ±
3.89 µs ± 35.6 ns per loop (mean
```

Fast computation examples

```
a=np.arange(10)
a2=a*2
print(a)
print(a2)
```

```
[0 1 2 3 4 5 6 7 8 9]
[ 0  2  4  6  8 10 12 14 16 18]
```

- Because each iteration in NumPy is using identical operations only the data differs, this can be compiled into machine language, and then performed in one go, hence avoiding Python's dynamic typing.

- The name vector operation comes from linear algebra
  - addition of two vectors $\mathrm{a} = [a_1,\ a_2]$, $\mathrm{b} = [b_1,\ b_2]$ is element-wise addition

$$\mathrm{a} + \mathrm{b} = [a_1 + b_1,\ a_2 + b_2]$$

Arithmetic Operations in NumPy

- The basic arithmetic operations in NumPy are defined in the vector form.
  - +: addition
  - −: subtraction
  - *: multiplication
  - /: division
  - //:floor division
  - **: power
  - %: remainder

```
# Arithmetic Operations in NumPy
a = np.arange(1, 10)
b = np.arange(2, 20, 2)
print(a)
print(b)
print("a + b:")
print(a+b)
print("a - b:")
print(a-b)
print("a * b:")
print(a*b)
print("a / b:")
print(a/b)
print("a // b:")
print(a//b)
print("a**b:")
print(a**b)
print("a%b:")
print(a%b)
```

```
[1 2 3 4 5 6 7 8 9]
[ 2  4  6  8 10 12 14 16 18]
a + b:
[ 3  6  9 12 15 18 21 24 27]
a - b:
[-1 -2 -3 -4 -5 -6 -7 -8 -9]
a * b:
[  2   8  18  32  50  72  98 128 162]
a / b:
[0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5]
a // b:
[0 0 0 0 0 0 0 0 0]
a**b:
[          1          16         729
  -381759919           0 -1953380655]
a%b:
[1 2 3 4 5 6 7 8 9]
```

Aggregations: max, min, sum, mean, standard deviations…

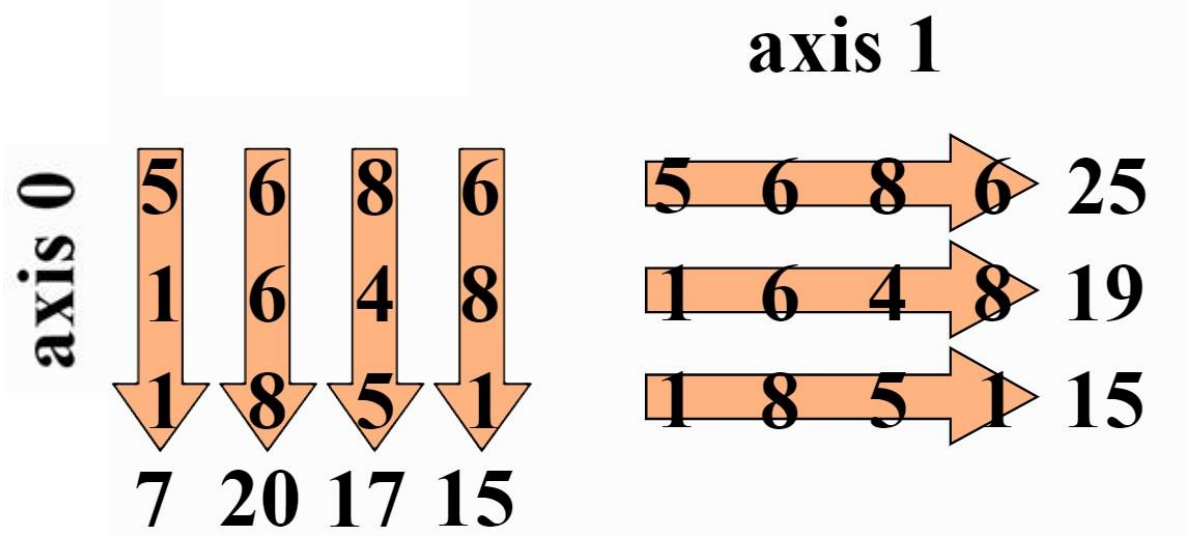
- Aggregations allow us to describe the information in an array by using few numbers

```
# aggregation
np.random.seed(0)
a=np.random.randint(-100, 100, (4,5))
print(a)
print(f"Minimum: {a.min()}, maximum: {a.max()}")
print(f"Sum: {a.sum()}")
print(f"Mean: {a.mean()}, standard deviation: {a.std()}")
```

```
[[ 72 -53  17  92 -33]
 [ 95   3 -91 -79 -64]
 [-13 -30 -12  40 -42]
 [ 93 -61 -13  74 -12]]
Minimum: -91, maximum: 95
Sum: -17
Mean: -0.85, standard deviation: 58.39886557117355
```

Aggregation over certain axes

- Instead of aggregating over the whole array, we can aggregate over certain axes only as well.




```
b=np.array([[5, 6, 8, 6],
            [1, 6, 4, 8],
            [1, 8, 5, 1]])
print(b)
print("Column sums:", b.sum(axis=0))
print("Row sums:", b.sum(axis=1))
```

```
[[5 6 8 6]
 [1 6 4 8]
 [1 8 5 1]]
Column sums: [ 7 20 17 15]
Row sums: [25 19 15]
```

Python function, NumPy function, NumPy Method

Python function	NumPy function	NumPy method
sum	np.sum	a.sum
•	np.prod	a.prod
•	np.mean	a.mean
•	np.std	a.std
•	np.var	a.var
min	np.min	a.min
max	np.max	a.max

- Most of the aggregation functions in NumPy have corresponding methods.
- Python language has builtin functions sum, min, max, etc.
- Do not accidentally use Python built-in functions for arrays, since they will be significantly slower than NumPy's functions and methods.

Efficiency of NumPy functions

```
a=np.arange(1000)
%timeit np.sum(a)

%timeit sum(a)
```

```
7.12 µs ± 187 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)
191 µs ± 1.56 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)
```

- The speed of NumPy partly comes from the fact that its arrays must have same type for all the elements. This requirement allows some efficient optimizations.

Broadcasting

- We have seen that NumPy allows array operations that are performed element-wise.
- NumPy also allows binary operation that do not require the two arrays to have the same shape.
- E.g., add 4 to all elements of an array

```
np.arange(3) + 4
```

```
array([4, 5, 6])
```

Broadcasting

- How binary operation is performed?
- NumPy tries to stretch the arrays to have the same shape, then perform the element-wise operation.
- In NumPy, this stretching is called broadcasting.

```
np.arange(3) + 4
```

```
array([4, 5, 6])
```

NumPy first stretched the scalar 4 to the array `np.array([4,4,4])` and then performed the element-wise addition.

Broadcasting

```
a=np.full((3,3), 5)
b=np.arange(3)
print("a:", a, sep="\n")
print("b:", b)
print("a+b:", a+b, sep="\n")
```

```
a:
[[5 5 5]
 [5 5 5]
 [5 5 5]]
b: [0 1 2]
a+b:
[[5 6 7]
 [5 6 7]
 [5 6 7]]
```

- In this example the second argument `b` was first broadcasted to the array

```
np.array([[0, 1, 2],
          [0, 1, 2],
          [0, 1, 2]])
```

- Then the addition was performed.

Comparisons and Masking

- Just like NumPy allows element-wise arithmetic operations between arrays, it is also possible to compare two arrays element-wise.

- We can also count the number of comparisons that were True. This solution relies on the interpretation that True corresponds to 1 and False corresponds to 0.

- Broadcasting rules also apply to comparison

```
a=np.array([1,3,4])
b=np.array([2,2,7])
c = a < b
print(c)
```

```
[ True False  True]
```

```
print(np.sum(c))
```

```
2
```

```
print(a > 0)
```

```
[ True  True  True]
```

Masking

- Another use of `Boolean` arrays is that they can be used to select a subset of elements. It is called *masking*.

- It can also be used to assign a new value. For example, the following zeroes out the negative numbers.

```
a = np.random.randint(-2, 10, (3,4))
print("a: ")
print(a)
c = a > 0
print("Positive numbers in a:")
print(a[c])      # Select only the positive numbers

a[~c] = 0
print("zeroes out negative numbers in a:")
print(a)
```

```
a:
[[ 3  4  6  2]
 [-1  2  7  8]
 [ 8  6 -1 -1]]
Positive numbers in a:
[3 4 6 2 2 7 8 8 6]
zeroes out negative numbers in a:
[[3 4 6 2]
 [0 2 7 8]
 [8 6 0 0]]
```

Fancy Indexing

- Using indexing we can get a single elements from an array. If we wanted multiple (not necessarily contiguous) elements, we would have to index several times.

```
# Fancy indexing
a=np.arange(10)
a2=np.array([a[2], a[5], a[7]])
print(a)
print(a2)
```

```
[0 1 2 3 4 5 6 7 8 9]
[2 5 7]
```

- That's quite verbose. *Fancy indexing* provides a concise syntax for accessing multiple elements.

```
a=np.arange(10)
idx=[2,5,7]
print(a[idx])
print(a[[2,5,7]])

# assign
a[idx] = 0
print(a)
```

```
[2 5 7]
[2 5 7]
[0 1 0 3 4 0 6 0 8 9]
```

- We can also assign to multiple elements through fancy indexing.

Fancy Indexing

- Fancy indexing works also for higher dimensional arrays.

```
b=np.arange(16).reshape(4,4)
print(b)
row=np.array([0,2])
col=np.array([1,3])
print(b[row, col])
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
[ 1 11]
```

- We can also combine normal indexing, slicing and fancy indexing.

```
b=np.arange(16).reshape(4,4)
print("b:")
print(b)
print("b[:,[0,3]]:")
print(b[:, [0, 3]])
```

```
b:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
b[:,[0,3]]:
[[ 0  3]
 [ 4  7]
 [ 8 11]
 [12 15]]
```

Sorting arrays

- Sorting one dimensional array is similar to sort a list.

- We can also sort a high dimensional array along different axes.

```
# sorting arrays
a=np.array([2,1,4,3,5])
print(np.sort(a))          # Does not modify the argument
print(a)

a.sort()             # Modifies the argument
print(a)
```

```
[1 2 3 4 5]
[2 1 4 3 5]
[1 2 3 4 5]
```

```
b=np.random.randint(0,10, (4,4))
print(b)

print(np.sort(b, axis=0))   # sort each column
print(np.sort(b, axis=1))   # Sort each row
```

```
[[7 9 9 3]
 [6 7 2 0]
 [3 5 9 4]
 [4 6 4 4]]
[[3 5 2 0]
 [4 6 4 3]
 [6 7 9 4]
 [7 9 9 4]]
[[3 7 9 9]
 [0 2 6 7]
 [3 4 5 9]
 [4 4 4 6]]
```

Sorting arrays

- A related operation is the `argsort` function. Which doesn't sort the elements but returns the indices of the sorted elements.

```
a=np.array([2,5,6,1,3])
print("Array a:", a)
idx = np.argsort(a)
print("Indices:", idx)
print(a[idx])
```

```
Array a: [2 5 6 1 3]
Indices: [3 0 4 1 2]
[1 2 3 5 6]
```

These indices [3, 0, 4, 1, 2] say that the smallest element of the array is in position 3 of a, second smallest elements is in position 0 of a, third smallest is in position 4, and so on.

Matrix operations

- NumPy support a wide variety of matrix operations, such as matrix multiplication, solve systems of linear equations, compute eigenvalues/eigenvectors, matrix decompositions and other linear algebra related operations.

Applications of NumPy

- Mathematics (MATLAB Replacement)
- Plotting (Matplotlib)
- Backend (Pandas, Digital Photography)
- Machine Learning
- Signal Processing